

HW 07 - Bike rentals in DC

Max Asmar

Bike sharing systems are new generation of traditional bike rentals where whole process from membership, rental and return back has become automatic. Through these systems, user is able to easily rent a bike from a particular position and return back at another position. Currently, there are about over 500 bike-sharing programs around the world which is composed of over 500 thousands bicycles. Today, there exists great interest in these systems due to their important role in traffic, environmental and health issues.

Apart from interesting real world applications of bike sharing systems, the characteristics of data being generated by these systems make them attractive for the research. Opposed to other transport services such as bus or subway, the duration of travel, departure and arrival position is explicitly recorded in these systems. This feature turns bike sharing system into a virtual sensor network that can be used for sensing mobility in the city. Hence, it is expected that most of important events in the city could be detected via monitoring these data.

Source: UCI Machine Learning Repository - Bike Sharing Dataset

Getting started

Open the `hw-07-bike-rentals-dc.rmd` file in your RStudio Notebook on JupyterHub. Knit the document to make sure it compiles without errors.

Learning Objectives

In this assignment, you will practice:

- Recoding variables into factors with meaningful levels
- Creating new variables through calculations
- Validating data integrity
- Creating effective data visualizations
- Fitting and interpreting linear regression models
- Working with interaction terms in models

Warm up

Before we introduce the data, let's warm up with some simple exercises. Update the YAML of your R Markdown file with your information, knit, commit, and push your changes. Make sure to commit with a meaningful commit message. Then, go to your repo on GitHub and



Figure 1: Photo by Viktor Kern on Unsplash

confirm that your changes are visible in your Rmd **and** md files. If anything is missing, commit and push again.

Packages

We'll use the **tidyverse** package for much of the data wrangling and visualisation and the data lives in the **dsbox** package. These packages are already installed for you. You can load them by running the following in your Console:

```
library(tidyverse)
library(dsbox)
library(tidymodels)
```

Data

The data can be found in the **dsbox** package, and it's called **dcbikeshare**. Since the dataset is distributed with the package, we don't need to load it separately; it becomes available to us when we load the package. You can find out more about the dataset by inspecting its documentation, which you can access by running **?dcbikeshare** in the Console or using the Help menu in RStudio to search for **dcbikeshare**. You can also find this information [here](#).

The data include daily bike rental counts (by members and casual users) of Capital Bikeshare in Washington, DC in 2011 and 2012 as well as weather information on these days. The original data sources are <http://capitalbikeshare.com/system-data> and <http://www.freemeteo.com>.

Helpful Functions for This Assignment

Factor Recoding: - `factor(variable, levels = c(...), labels = c(...))` - Convert to factor with specific levels and labels - The first level specified becomes the baseline (reference) level

Creating New Variables: - `mutate(new_var = old_var * constant)` - Create calculated variables

Validation: - `all(condition)` - Check if all values meet a condition - Logical operators: `==` (equal), `!=` (not equal)

Modeling: - `lm(y ~ x, data = data)` - Fit a linear model - `summary(model)` - View model output including coefficients and R^2 - `y ~ x1 + x2` - Multiple predictors - `y ~ x1 * x2` - Include interaction between `x1` and `x2` - Adjusted R^2 is found in the model summary output

Tip: For questions 1-6, save your recoded/calculated variables back to the dataset so you can use them in later questions.

*Exercises**Data wrangling*

1. Recode the `season` variable to be a factor with meaningful level names as outlined in the codebook, with spring as the baseline level.

```
dcbikeshare <- dcbikeshare %>%
  mutate(season = factor(season, levels = c(2, 3, 4, 1), labels = c("spring", "summer", "fall", "winter"))
```

2. Recode the binary variables `holiday` and `workingday` to be factors with levels no (0) and yes (1), with no as the baseline level.

```
dcbikeshare <- dcbikeshare %>%
  mutate(holiday = factor(holiday, levels = c(0, 1), labels = c("no", "yes")))
```

```
dcbikeshare <- dcbikeshare %>%
  mutate(workingday = factor(workingday, levels = c(0, 1), labels = c("no", "yes")))
```

3. Recode the `yr` variable to be a factor with levels 2011 and 2012, with 2011 as the baseline level.

```
dcbikeshare <- dcbikeshare %>%
  mutate(yr = factor(yr, levels = c(0, 1), labels = c("2011", "2012")))
```

4. Recode the `weathersit` variable as 1 - clear, 2 - mist, 3 - light precipitation, and 4 - heavy precipitation, with clear as the baseline.

```
dcbikeshare <- dcbikeshare %>%
  mutate(weathersit = factor(weathersit, levels = c(1, 2, 3, 4), labels = c("clear", "mist", "light pre
```

5. Calculate raw temperature, feeling temperature, humidity, and windspeed as their values given in the dataset multiplied by the maximum raw values stated in the codebook for each variable. Instead of writing over the existing variables, create new ones with concise but informative names.

```
dcbikeshare <- dcbikeshare %>%
  mutate(rawtemperature = temp*41)
```

```
dcbikeshare <- dcbikeshare %>%
  mutate(feeltemperature = atemp*50)
```

```
dcbikeshare <- dcbikeshare %>%
  mutate(humidity = hum*100)
```

```
dcbikeshare <- dcbikeshare %>%
  mutate(rawwindspeed = windspeed*67)
```

6. Check that the sum of `casual` and `registered` adds up to `cnt` for each record. This is a data validation step to ensure the dataset is internally consistent.

```
dcbikeshare %>%
  mutate(check_cnt = (casual + registered == cnt)) %>%
  summarize(all_correct = all(check_cnt))

## # A tibble: 1 x 1
##   all_correct
##   <lgl>
## 1 TRUE
```

One way to do this is to create a new logical variable that checks if `casual + registered == cnt` for each row, then verify that all values are TRUE.

Write your answer: Do the casual and registered counts sum to the total count for all records? (Yes/No)

Yes, the casual and registered counts sum to the total count for all records. I know this because the tibble outputs true for `all_correct` variable, which checks the sum of casual and registered variables.

Checkpoint: Before moving on to exploratory data analysis and modeling, verify your data wrangling:

- Run `str(dcbikeshare)` to check that your factor variables show the correct levels

```
str(dcbikeshare)

## tibble [731 x 20] (S3: tbl_df/tbl/data.frame)
##  $ instant      : num [1:731] 1 2 3 4 5 6 7 8 9 10 ...
##  $ dteday       : Date[1:731], format: "2011-01-01" "2011-01-02" ...
##  $ season      : Factor w/ 4 levels "spring","summer",...: 4 4 4 4 4 4 4 4 4 4 ...
##  $ yr          : Factor w/ 2 levels "2011","2012": 1 1 1 1 1 1 1 1 1 1 ...
##  $ mnth        : num [1:731] 1 1 1 1 1 1 1 1 1 1 ...
##  $ holiday     : Factor w/ 2 levels "no","yes": 1 1 1 1 1 1 1 1 1 1 ...
##  $ weekday     : num [1:731] 6 0 1 2 3 4 5 6 0 1 ...
##  $ workingday  : Factor w/ 2 levels "no","yes": 1 1 2 2 2 2 2 1 1 2 ...
##  $ weathersit   : Factor w/ 4 levels "clear","mist",...: 2 2 1 1 1 1 2 2 1 1 ...
##  $ temp       : num [1:731] 0.344 0.363 0.196 0.2 0.227 ...
##  $ atemp      : num [1:731] 0.364 0.354 0.189 0.212 0.229 ...
##  $ hum       : num [1:731] 0.806 0.696 0.437 0.59 0.437 ...
##  $ windspeed  : num [1:731] 0.16 0.249 0.248 0.16 0.187 ...
##  $ casual     : num [1:731] 331 131 120 108 82 88 148 68 54 41 ...
##  $ registered : num [1:731] 654 670 1229 1454 1518 ...
##  $ cnt       : num [1:731] 985 801 1349 1562 1600 ...
```

```
## $ rawtemperature : num [1:731] 14.11 14.9 8.05 8.2 9.31 ...
## $ feeltemperature: num [1:731] 18.18 17.69 9.47 10.61 11.46 ...
## $ humidity       : num [1:731] 80.6 69.6 43.7 59 43.7 ...
## $ rawwindspeed   : num [1:731] 10.7 16.7 16.6 10.7 12.5 ...
```

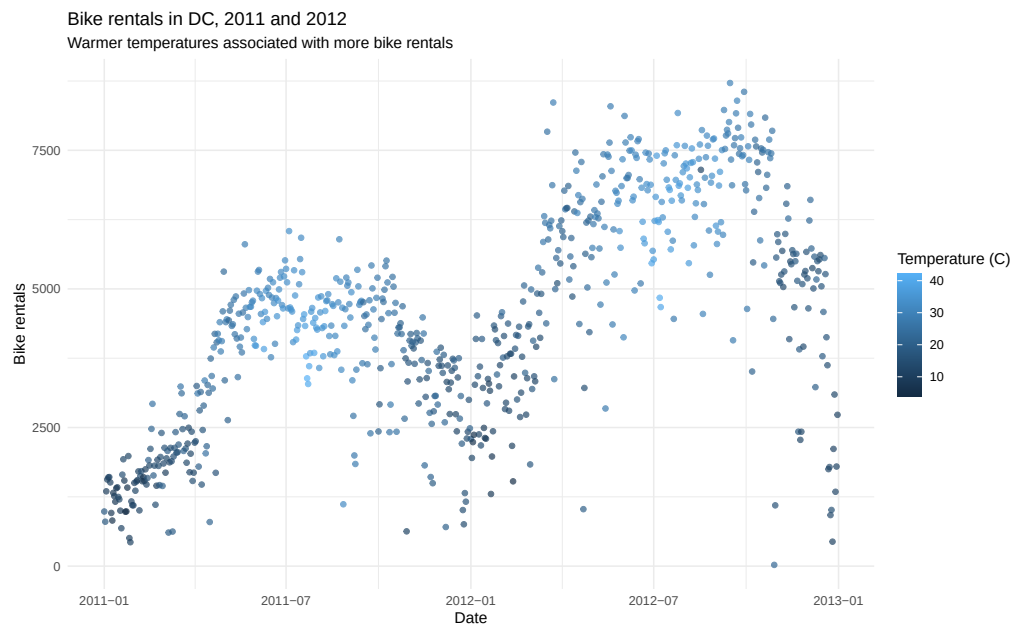
- Check that your new calculated variables (raw temperature, humidity, etc.) have reasonable values
- Verify that all validations passed

If something looks wrong, revisit your data wrangling code before continuing.

Knit, *commit*, and *push* your changes to *GitHub* with an appropriate commit message. Make sure to commit and push all changed files so that your *Git* pane is cleared up afterwards.

Exploratory data analysis

- The code below creates a visualization showing the relationship between date, bike rentals, and feeling temperature. Examine the code and visualization, then answer the questions.



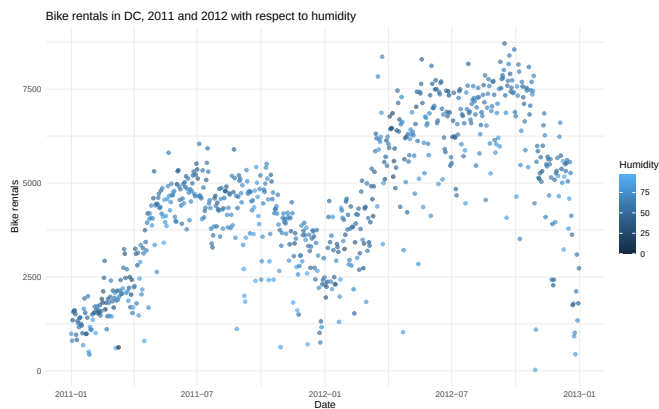
- What does this visualization reveal about the relationship between temperature and bike rentals over time?

The visualization reveals that higher temperatures are associated with more bike rentals, while lower temperatures are associated with

fewer bike rentals. Most of these higher temperatures appear to be during June, July, and August, while most of the lower temperatures tend to be January, February, and Winter.

- b. Modify the code to color the points by humidity instead of temperature. What new pattern (if any) emerges?

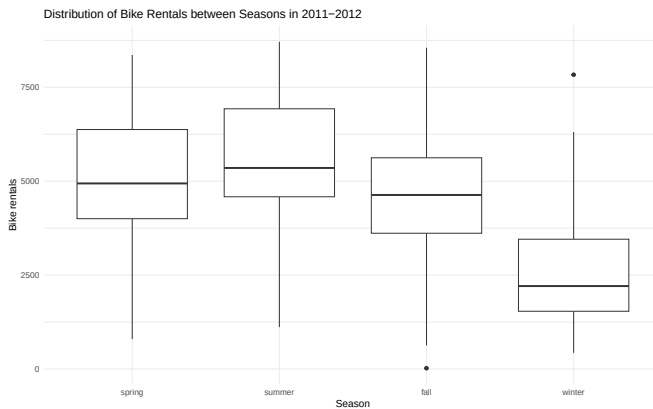
```
dcbikeshare %>%
  mutate(humidity = hum * 100) %>%
  ggplot(mapping = aes(x = dteday, y = cnt, color = humidity)) +
  geom_point(alpha = 0.7) +
  labs(
    title = "Bike rentals in DC, 2011 and 2012 with respect to humidity",
    x = "Date",
    y = "Bike rentals",
    color = "Humidity"
  ) +
  theme_minimal()
```



There is no association between humidity and bike rentals, looking at the graph, as there is a similar level of humidity throughout all months of the year.

8. Create a visualization displaying the relationship between bike rentals and season. Interpret the plot in context of the data.

```
dcbikeshare %>%
  ggplot(mapping = aes(x = season, y = cnt)) +
  geom_boxplot() +
  labs(
    title = "Distribution of Bike Rentals between Seasons in 2011-2012",
    x = "Season",
    y = "Bike rentals",
  ) +
  theme_minimal()
```



The boxplot shows that summer was the season with the highest median bike rentals, while winter was the season with the lowest median bike rentals. Fall and spring tended to have a similar number of median bike rentals, and the spread for all seasons was about equal.

Knit, commit, and push your changes to GitHub with an appropriate commit message. Make sure to commit and push all changed files so that your Git pane is cleared up afterwards.

Modeling

- Fit a linear model predicting total daily bike rentals from daily temperature. Write the linear model, interpret the slope and the intercept in context of the data, and determine and interpret the R^2 .

```
# Remember to change eval=FALSE to eval=TRUE!
cnt_rawtemperature_fit <- linear_reg() %>%
  set_engine("lm") %>%
  fit(cnt ~ rawtemperature, data = dcbikeshare)

tidy(cnt_rawtemperature_fit)

## # A tibble: 2 x 5
##   term          estimate std.error statistic  p.value
##   <chr>          <dbl>    <dbl>    <dbl>    <dbl>
## 1 (Intercept)    1215.    161.      7.54 1.43e-13
## 2 rawtemperature    162.     7.44    21.8 2.81e-81

glance(cnt_rawtemperature_fit)

## # A tibble: 1 x 12
##   r.squared adj.r.squared sigma statistic  p.value    df logLik   AIC   BIC
##   <dbl>      <dbl> <dbl>    <dbl>    <dbl> <dbl> <dbl> <dbl> <dbl>
## 1   0.394      0.393 1509.    473. 2.81e-81     1 -6387. 12780. 12793.
## # i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

The linear model is $\text{cnt} = 1215 + 162 * \text{rawtemperature}$

The slope means that for every 1 degree celsius increase in temperature, the number of bike rentals is expected to increase by 162, on average. The intercept means that when the temperature is 0 degrees Celsius, the number of bike rentals is expected to be 1215. The R^2 value of 0.394 means that approximately 39.4% of the variability in number of bike rentals is explained by temperature.

10. Fit another linear model predicting total daily bike rentals from daily feeling temperature. Write the linear model, interpret the slope and the intercept in context of the data, and determine and interpret the R^2 . Is temperature or feeling temperature a better predictor of bike rentals? Explain your reasoning.

```
# Remember to change eval=FALSE to eval=TRUE!
cnt_feeltemperature_fit <- linear_reg() %>%
  set_engine("lm") %>%
  fit(cnt ~ feeltemperature, data = dc_bikeshare)

tidy(cnt_feeltemperature_fit)

## # A tibble: 2 x 5
##   term          estimate std.error statistic  p.value
##   <chr>          <dbl>    <dbl>    <dbl>    <dbl>
## 1 (Intercept)    946.    171.     5.52 4.67e- 8
## 2 feeltemperature 150.     6.83    22.0 1.85e-82

glance(cnt_feeltemperature_fit)

## # A tibble: 1 x 12
##   r.squared adj.r.squared sigma statistic  p.value    df logLik    AIC    BIC
##   <dbl>      <dbl> <dbl>    <dbl>    <dbl> <dbl> <dbl> <dbl> <dbl>
## 1   0.398      0.397 1504.    482. 1.85e-82     1 -6384. 12774. 12788.
## # i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

The linear model is $\text{cnt} = 946 + 150 * \text{feeltemperature}$

The slope means that for every 1 degree celsius increase in apparent temperature, the number of bike rentals is expected to increase by 150, on average. The intercept means that when the apparent temperature is 0 degrees Celsius, the number of bike rentals is expected to be 946. The R^2 value of 0.398 means that approximately 39.8% of the variability in number of bike rentals is explained by apparent temperature.

Comparing the R^2 value of the apparent temperature model and the actual temperature model, apparent temperature is a better predictor of bike rentals, because it describes more of the variability in

bike rentals. Apparent temperature describes around 0.2% more of the variability than actual temperature.

11. Fit a model predicting total daily bike rentals from season, year, whether the day is holiday or not, whether the day is a workingday or not, the weather category, temperature, feeling temperature, humidity, and windspeed, as well as the interaction between feeling temperature and holiday. Record adjusted R^2 of the model.

```
# Remember to change eval=FALSE to eval=TRUE!
cnt_multiplevariable_fit <- linear_reg() %>%
  set_engine("lm") %>%
  fit(cnt ~ season + yr + holiday + workingday + weathersit + rawtemperature + feeltemperature + humidity)

tidy(cnt_multiplevariable_fit)

## # A tibble: 14 x 5
##   term                                estimate std.error statistic    p.value
##   <chr>                                <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)                        2715.      269.      10.1  1.48e- 22
## 2 seasonsummer                       -277.      100.      -2.76  5.93e- 3
## 3 seasonfall                          410.       96.2       4.26  2.30e- 5
## 4 seasonwinter                      -1131.      114.      -9.96  5.80e- 22
## 5 yr2012                             2014.       61.7      32.6  6.38e-144
## 6 holidayyes                        -1384.      495.      -2.79  5.34e- 3
## 7 workingdayyes                       120.       67.9       1.76  7.83e- 2
## 8 weathersitmist                      -420.       81.3      -5.17  3.04e- 7
## 9 weathersitlight precipitation -1907.      208.      -9.19  4.21e- 19
## 10 rawtemperature                     103.       34.0       3.03  2.55e- 3
## 11 feeltemperature                     18.8       30.5       0.616 5.38e- 1
## 12 humidity                          -13.6       2.96      -4.60  5.09e- 6
## 13 rawwindspeed                      -40.6       6.49      -6.26  6.59e- 10
## 14 holidayyes:feeltemperature         34.4       20.6       1.67  9.54e- 2

glance(cnt_multiplevariable_fit)

## # A tibble: 1 x 12
##   r.squared adj.r.squared sigma statistic    p.value    df logLik    AIC    BIC
##   <dbl>      <dbl> <dbl>     <dbl>    <dbl> <dbl> <dbl> <dbl> <dbl>
## 1    0.823    0.820  821.     257.  1.63e-259    13 -5936. 11902. 11971.
## # i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

The adjusted r^2 value is 0.82, meaning that approximately 82% of the variability in number of bike rentals is explained by the model above.

12. Your model in Question 11 includes an interaction between feeling temperature and holiday. Write out the linear model equation for holidays and the linear model equation for non-holidays separately (substitute the appropriate values for the holiday variable). Is the slope of temperature the same or different for these two models? How about the slope for feeling temperature? Explain why based on the interaction term.

Question 11 equation:

$$\text{cnt} = 2715 - 277 * \text{seasonsummer} + 410 * \text{seasonfall} - 1131 * \text{seasonwinter} + 2014 * \text{yr2012} - 1384 * \text{holidayyes} + 120 * \text{workingdayyes} - 420 * \text{weathersitmist} - 1907 * \text{weathersitlight} \text{precipitation} + 103 * \text{raw temperature} + 18.8 * \text{feeltemperature} - 13.6 * \text{humidity} - 40.6 * \text{rawwindspeed} + 34.4 * \text{holidayyes:feeltemperature}$$

I substituted 1 for the holidayyes variable.

Holiday equation:

$$\text{cnt} = 1331 - 277 * \text{seasonsummer} + 410 * \text{seasonfall} - 1131 * \text{seasonwinter} + 2014 * \text{yr2012} + 120 * \text{workingdayyes} - 420 * \text{weathersitmist} - 1907 * \text{weathersitlight} \text{precipitation} + 103 * \text{raw temperature} + 53.2 * \text{feeltemperature} - 13.6 * \text{humidity} - 40.6 * \text{rawwindspeed}$$

I substituted 0 for the holidayyes variable.

Non-Holiday Equation:

$$\text{cnt} = 2715 - 277 * \text{seasonsummer} + 410 * \text{seasonfall} - 1131 * \text{seasonwinter} + 2014 * \text{yr2012} + 120 * \text{workingdayyes} - 420 * \text{weathersitmist} - 1907 * \text{weathersitlight} \text{precipitation} + 103 * \text{raw temperature} + 18.8 * \text{feeltemperature} - 13.6 * \text{humidity} - 40.6 * \text{rawwindspeed}$$

The slope of temperature is the same for both models, because the interaction term does not include temperature. However, the feeling temperature slope is different for each model: The feeling temperature has a coefficient of 53.2 when it is a holiday, while the feeling temperature has a coefficient of 18.8 when it is not a holiday. This is because the interaction term depends on the feeling temperature, which causes the weight of feeling temperature to shift.

13. Interpret the slopes of season and feeling temperature in the context of bike rentals. If the slopes are different for holidays and non-holidays, make sure to interpret both. If the variable has multiple levels, make sure you interpret all of the slope coefficients associated with it.

The slope of the feeling temperature in the holiday equation means that for every 1 degree Celsius increase in apparent temperature, given that it is a holiday, the number of bike rentals is expected to increase by 53.2, with all else held constant.

The slope of the feeling temperature in the non-holiday equation means that for every 1 degree Celsius increase in apparent temper-

ature, given that it is not a holiday, the number of bike rentals is expected to increase by 18.8, with all else held constant.

The slope of the seasonsummer variable is the same for both the holiday and non-holiday equations. The slope of the seasonsummer variable means that if it is summer, the number of bike rentals is expected to decrease by 277 compared to Spring, with all else held constant.

The slope of the seasonfall variable is the same for both the holiday and non-holiday equations. The slope of the seasonfall variable means that if it is fall, the number of bike rentals is expected to increase by 410 compared to Spring, with all else held constant.

The slope of the seasonwinter variable is the same for both the holiday and non-holiday equations. The slope of the seasonwinter variable means that if it is winter, the number of bike rentals is expected to decrease by 1131 compared to Spring, with all else held constant.

14. Interpret the intercept in the context of bike rentals. If the intercept is different for holidays and non-holidays, make sure to interpret both.

In the holiday equation, the intercept means that when it is Spring in 2011, and it is a holiday, and it is not a working day, when there is no mist or light precipitation, and there is no humidity or windspeed, and the actual and feel temperatures are 0, the expected number of bike rentals is 1331. However, if all of these conditions are true but it is not a holiday, then the expected number of bike rentals is 2715.

15. According to this model, assuming everything else is the same, in which season does the model predict total daily bike rentals to be highest and which to be the lowest?

According to the model, the model predicts that daily bike rentals will be the greatest in the fall, while the daily bike rentals are predicted to be the least in the winter. I figured this out by comparing each season's slope. For example, summer has a slope of -277, spring has a slope of 0, fall has a slope of 410, and winter has a slope of -1131.

Knit, commit, and push your changes to GitHub with an appropriate commit message. Make sure to commit and push all changed files so that your Git pane is cleared up afterwards and review the md document on GitHub to make sure you're happy with the final state of your work.

Now go back through your write up to make sure you've answered all questions and all of your R chunks are properly labeled.

Once you decide that you are done with the homework, choose the knit drop down and select **Knit to tufte_handout** to generate a pdf.

Download and submit that pdf to Canvas.

Common Errors and Troubleshooting

Factor recoding issues: - **Error: “object not found”** after recoding → You didn’t save the changes back to the dataset. Use
`dcbikeshare <- dcbikeshare %>% mutate(...)`

- **Baseline level is wrong** → The first level you specify in `factor()` becomes the baseline
- **Factor levels don’t match codebook** → Check spelling and capitalization in your `labels` argument

Modeling errors: - **Error: “object ‘...’ not found”** in `lm()` → Make sure you recoded/created all variables first and saved them to the dataset

- **Can’t find R^2 or Adjusted R^2** → Run `summary(model_name)` not just the model name
- **Interaction term not showing** → Use `*` not `+` between the variables you want to interact (e.g., `atemp_raw * holiday`)

Interpretation issues: - **Don’t know which coefficient is which** → Factor variables show as `variablenameLevel` (e.g., `seasonwinter` means winter compared to baseline)

- **Confused about interaction terms** → The interaction coefficient modifies the slope - plug in values for the interacting variable to see how slopes change
- **Intercept doesn’t make sense** → Remember it’s when all predictors = 0 or baseline levels, which may not be realistic

Data wrangling: - **Calculated values seem wrong** → Double-check the multipliers from the codebook

- **Validation check fails** → Check for typos in variable names or logic operators